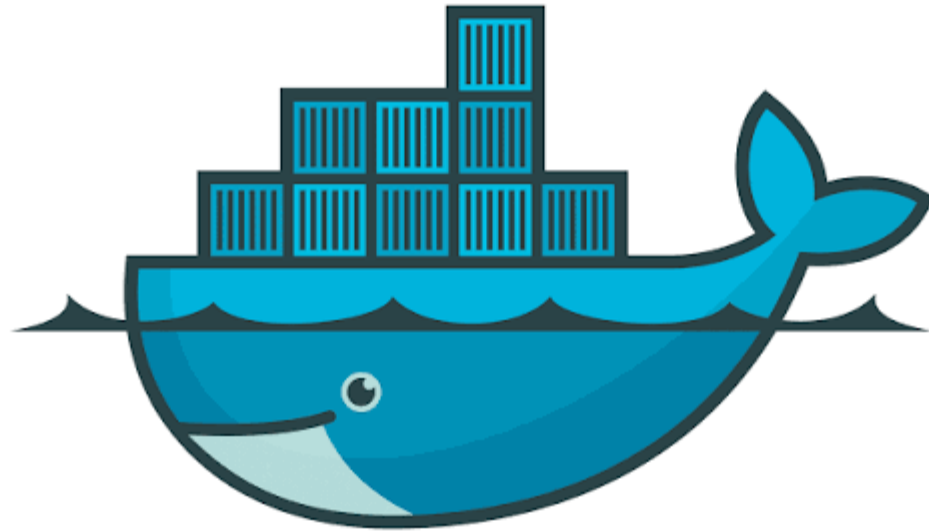


DevOps



Docker

Virtualization

Virtualization is a technology that allows one physical computer to run multiple virtual computers, called Virtual Machines (VMs).

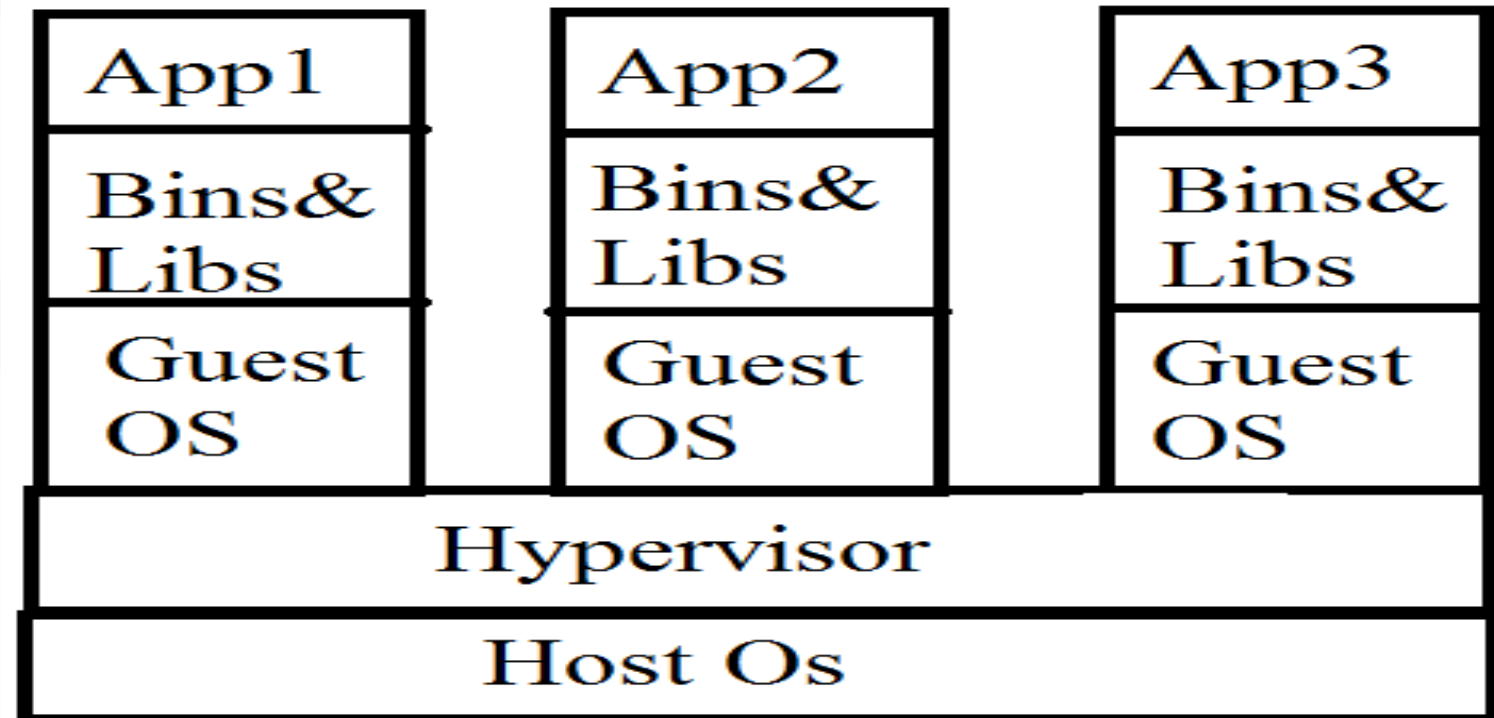
Each VM can have its own:

- Operating System
- CPU allocation
- RAM allocation
- Storage
- Applications

A software layer called a **hypervisor** manages the virtual machines

Host OS : The Host OS is the operating system running on the physical machine.

Guest OS : The operating system running inside a virtual machine is called the Guest OS.



Virtualization – Run Multiple Operating Systems on One Physical Machine

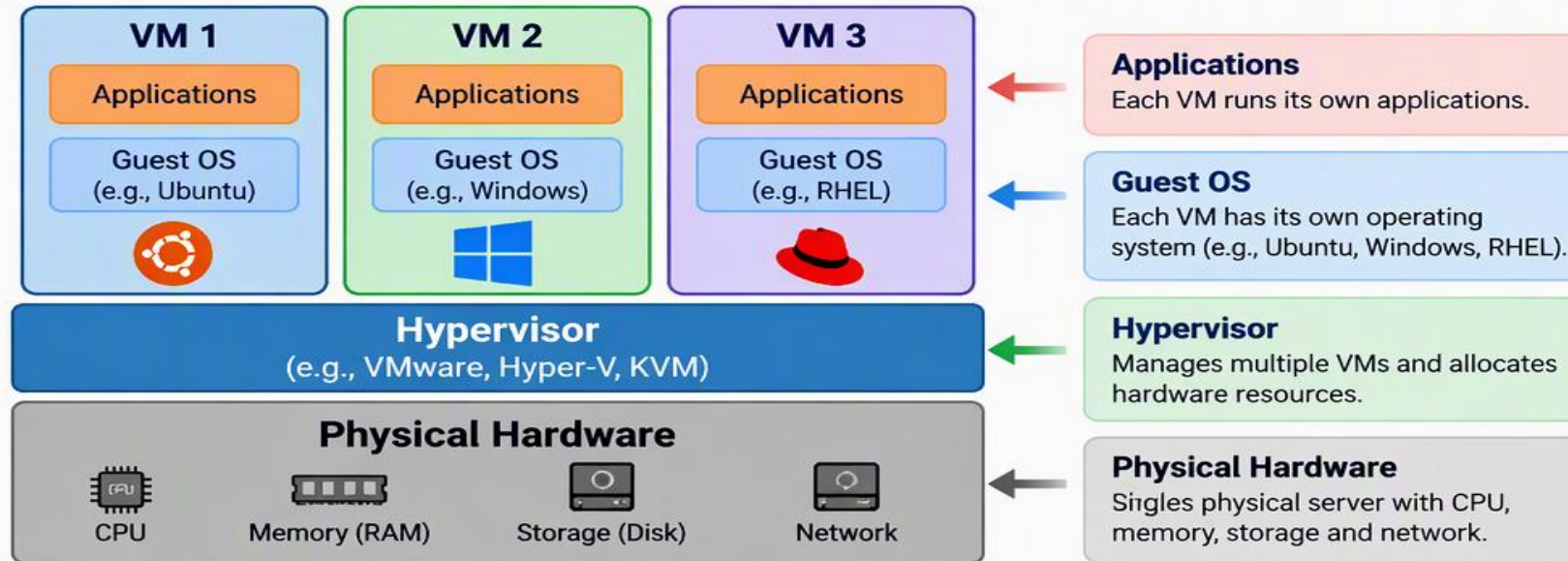
What is Virtualization?

Virtualization is a technology that allows one physical computer to run multiple virtual computers (Virtual Machines), each with its own operating system and applications. A software layer called a **hypervisor** manages the virtual machines and shares the underlying hardware resources (CPU, memory, storage, etc.).

Key Terms

- **Host OS:** The operating system running on the physical machine (for Type 2 hypervisors).
- **Guest OS:** The operating system running inside a virtual machine.
- **Hypervisor:** Software that creates and manages virtual machines.
- **Virtual Machine (VM):** An isolated environment with its own OS, resources and applications, running on the same physical hardware.

How Virtualization Works



Advantages of Virtualization

- **Eliminates the need for extra hardware**
Run multiple VMs on a single physical server.
- **Multiple OS on the same machine**
Run different operating systems (Linux, Windows, etc.) at the same time.
- **Easy maintenance and recovery**
VMs can be backed up, cloned, and quickly restored (using snapshots).
- **Lower cost of ownership**
Reduces hardware, power, cooling and data center costs.

Real-world Example



Types of Hypervisor

Type 1 (Bare Metal)

- Runs directly on the hardware.
- Common in enterprises and data centers.
- Examples: VMware ESXi, Microsoft Hyper-V.

Type 2 (Hosted)

- Runs on top of a host operating system.
- Common for personal use, development and testing.
- Examples: VMware Workstation, Oracle VirtualBox.

VMware : is virtualization software that allows us to create and run virtual computers (Virtual Machines or VMs) on a physical computer.

Important terms :

Physical Machine → Your actual laptop/server.

Host OS → Operating system installed directly on the physical machine, such as Windows 11.

VMware → Virtualization software used to create and manage virtual machines.

Virtual Machine (VM) → A software-based computer created using VMware.

Guest OS → Operating system installed inside the VM, such as Ubuntu, Windows Server, or another Linux distribution.

Example : Suppose your laptop has Windows 11 installed.

You install VMware on Windows 11 and create an Ubuntu VM.

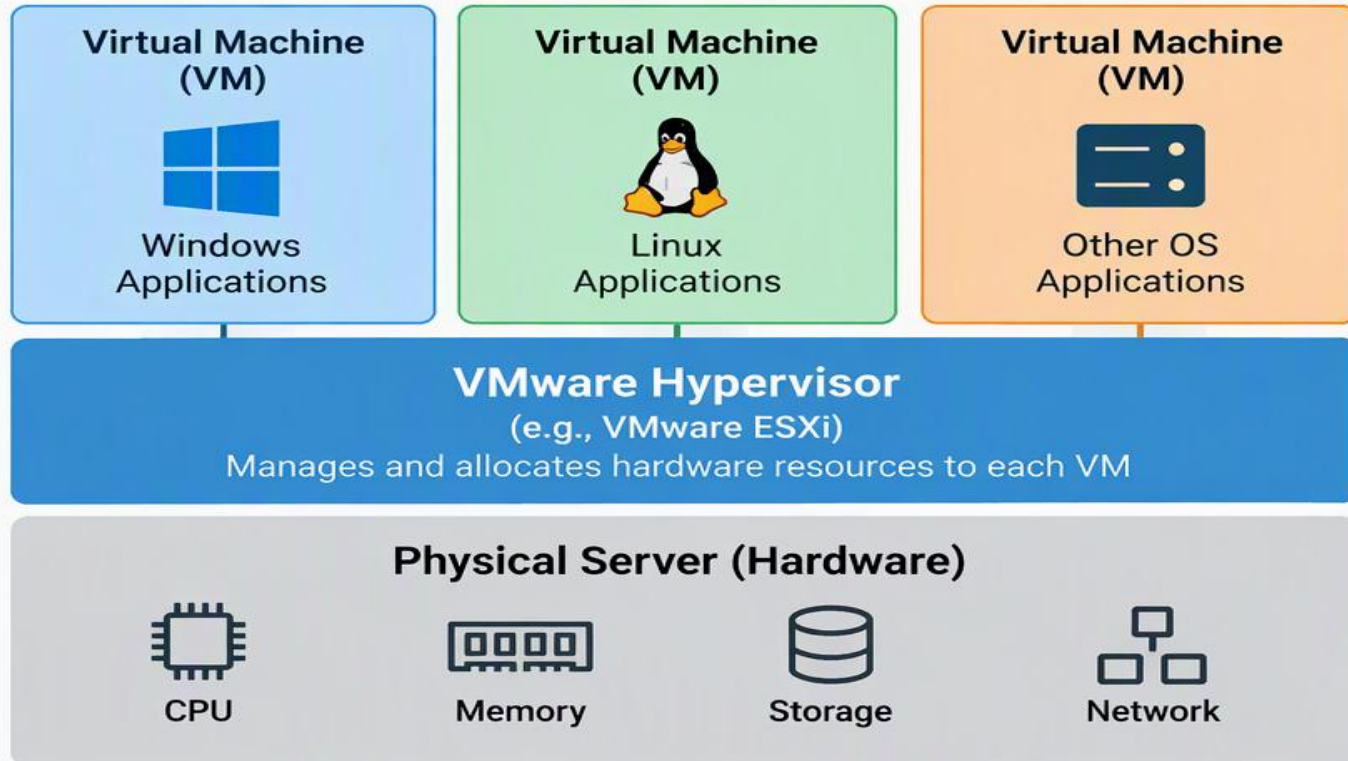


**Build a More Flexible, Efficient
and Secure IT Environment**

What is VMware?

VMware is a leading virtualization platform that allows you to run multiple virtual machines (VMs) on a single physical server. It abstracts the hardware and creates a software layer (hypervisor) so multiple operating systems can run independently, sharing the same physical resources.

How VMware Works



Key Benefits



Better Resource Utilization

Run multiple VMs on one physical server.



Cost Savings

Reduce hardware, power and maintenance costs.



Flexibility

Easily create, move, and manage virtual machines.



Improved Security

Isolated VMs reduce risk and enable safer testing.

Common Use Cases



Consolidating Servers

Run many workloads on fewer physical machines.



Test & Development

Quickly create isolated environments.



Disaster Recovery

Easily back up and restore virtual machines.



Hybrid Cloud

Integrate with VMware Cloud solutions for on-prem and cloud.

In short:

VMware helps organizations run, manage, and scale their IT environments more efficiently through virtualization.
One physical server. Many possibilities.

Disadvantages of Virtualization

Virtualization brings many benefits, but it also has some challenges and trade-offs to consider.



Performance Overhead

Virtual machines share the same physical hardware, which can lead to a slight performance overhead compared to running on dedicated hardware, especially for resource-intensive workloads.



Initial Cost

Setting up a virtualized environment requires investment in virtualization software, compatible hardware, and skilled IT staff. Licensing costs (for platforms like VMware) can also be significant.



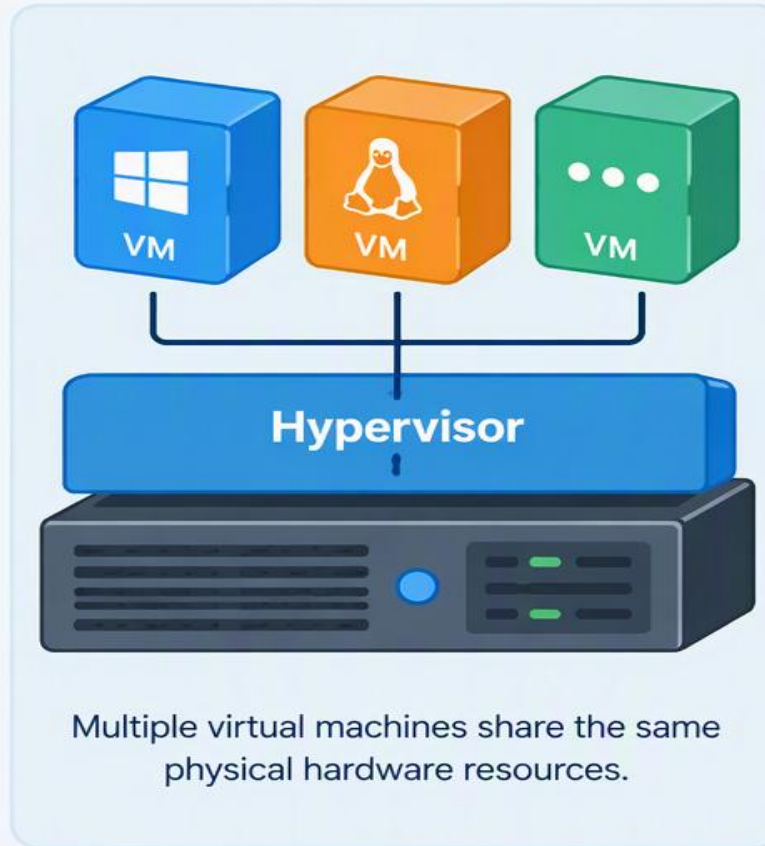
Increased Complexity

Managing a virtualized environment adds a layer of complexity. It requires specialized knowledge, tools, and ongoing management (e.g., monitoring, patching, capacity planning).



The Bottom Line

Virtualization is a powerful technology, but it's important to understand and plan for these disadvantages. With the right design, monitoring, and management, many of these challenges can be effectively mitigated.



Single Point of Failure

If the physical server (host) or the hypervisor fails, multiple virtual machines can be affected at the same time, potentially leading to wider downtime.



Security Risks

A vulnerability in the hypervisor could potentially compromise multiple virtual machines. VM sprawl and misconfigurations can also increase the attack surface.



Resource Contention

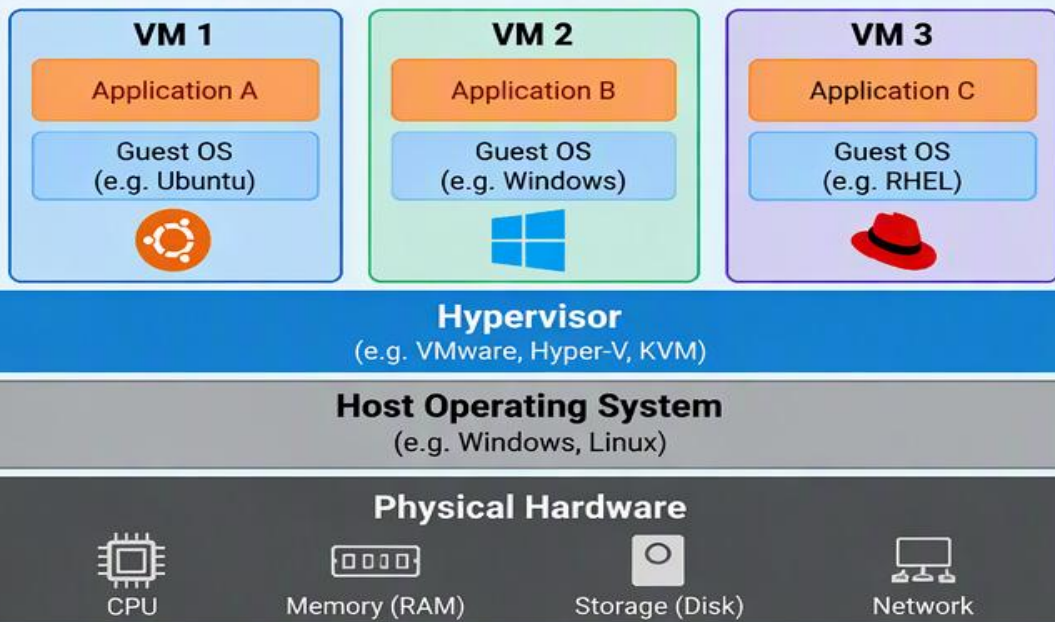
Virtual machines compete for the same CPU, memory, storage, and network resources. If one VM consumes too many resources, it can impact the performance of other VMs on the same host.

Virtualization vs Containerization

Two different ways to run applications. Understand the difference!

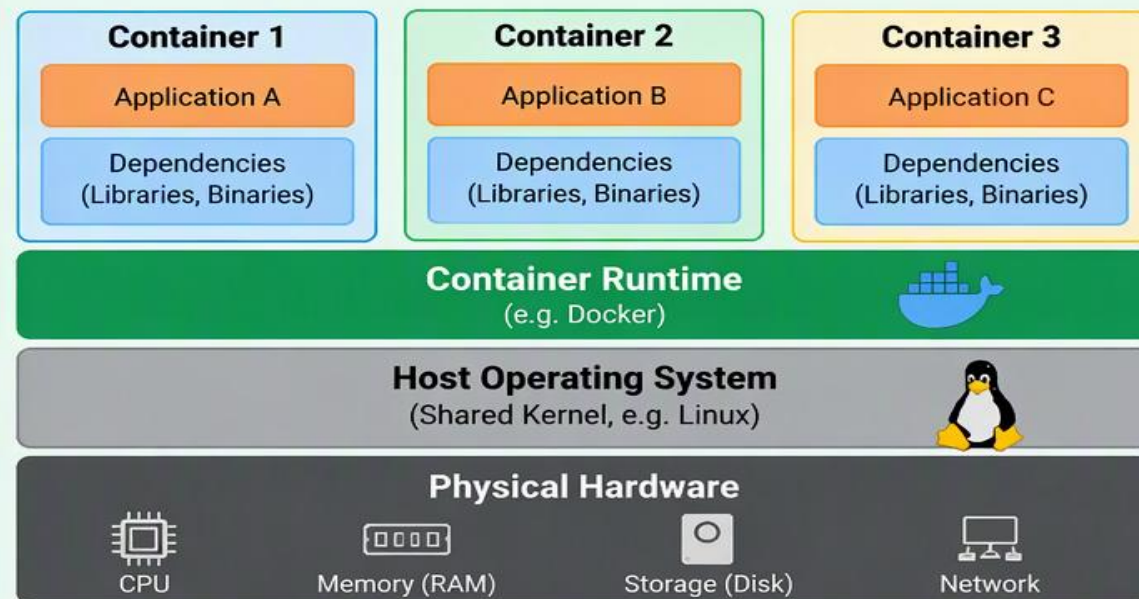
Virtualization (Using Virtual Machines)

Virtualization creates multiple virtual machines, each with its own operating system, on a single physical server.



Containerization (Using Containers)

Containerization shares the host operating system's kernel and runs applications in isolated containers. No separate guest OS is required.



Feature	Virtualization (VMs)	Containerization (Containers)
Operating System	Each VM has its own guest OS	Shares the host OS kernel (no guest OS)
Size	Larger (includes OS)	Smaller (only application and dependencies)
Resource Usage	Higher (separate RAM, CPU for each VM)	Lower (shares host resources)
Boot Time	Slower (needs to boot guest OS)	Faster (starts in seconds)
Isolation	Strong isolation (full OS)	Process-level isolation (lightweight)
Integration	Difficult to integrate VMs	Easy to integrate containers
Use Case	Running different operating systems, legacy apps	Microservices, modern applications, DevOps
Example Tools	VMware, Hyper-V, VirtualBox	Docker, containerd, Podman

Key Takeaways

- Use virtualization when you need different operating systems or strong isolation.
- Use containerization when you want lightweight, faster, and more efficient application deployment.
- Containers are not a replacement for VMs — they solve different problems.
- Both technologies are widely used in modern IT and cloud environments.



Basic Docker Commands

Manage Containers

- 1 Start a container**
Start an existing container.
`docker start <container_name>`

<container_name>
= name of the container
- 2 Go inside a container**
Open a shell inside a running container.
`docker attach <container_name>`

<container_name>
= name of the container
- 3 See all containers**
List all containers (running and stopped).
`docker ps -a`

-a → shows all containers
- 4 See only running containers**
List only running containers.
`docker ps`

Shows only running containers
- 5 Stop a container**
Stop a running container.
`docker stop <container_name>`

<container_name>
= name of the container
- 6 Delete a container**
Remove a container (stopped or running).
`docker rm <container_name>`

<container_name>
= name of the container

More Docker Commands

Images, Hub, and Containers



- 7 See all images**
List all Docker images present on your local machine.
`docker images`

Shows all local images
- 8 Find images in Docker Hub**
Search for an image in Docker Hub.
`docker search <image_name>`

<image_name>
= name to search (e.g. centos, ubuntu)
- 9 Download an image from Docker Hub**
Pull an image from Docker Hub to your local machine.
`docker pull <image_name>`

<image_name>
= name of the image (e.g. ubuntu, centos)
- 10 Give a name to a container**
Create and start a container with a custom name.
`docker run -it --name <container_name> <image_name>`

<container_name>
= container name
<image_name>
= image to use
- 11 Check if a service is running**
Check whether a Docker service (container) is running or not.
`docker ps`

Look at the STATUS column (e.g. Up or Exited)